

Scalable Technologies for Data Analysis

Serving

Davide Carneiro

Mestrado em Engenharia Informática

2025/2026

Serving

How can “gold” data be effectively used by the organization

- By the end of this class you should be able to
 - Explain why “gold” data is not automatically consumable, and what is required to turn data into a usable product
 - Distinguish between different serving patterns (SQL queries, materialized tables, APIs) and when each is appropriate
 - Understand the role of query engines (e.g., Trino) as a serving layer over a lakehouse, including their trade-offs
 - Decide between materialization vs. query-on-read, justifying choices based on latency, cost, and reuse
 - Recognize the need for a semantic layer and define metrics with consistent meaning and grain
 - Design simple data products (queries or APIs) with clear contracts and predictable performance
 - Apply a decision framework to choose the right serving strategy for a given use case in the project

The plan

1. Intro to modern Data Platforms and scalability
2. Contract-Driven Design and Data Flow Modeling
3. Storage + table formats
4. Processing patterns + optimization
5. Orchestration + production reliability
6. Serving: SQL/APIs/semantic layer 
7. Quality + observability + cost
8. Incremental + streaming
9. ML integration
10. Governance + evolution

Previously...

- In the previous classes, we
 - Designed data platforms around contracts, layers (bronze/silver/gold), and reproducible pipelines
 - Learned how to store and process data efficiently at scale, using formats, partitioning, and query optimization
 - Implemented reliable workflows with orchestration (Flyte), ensuring idempotency, retries, and backfills
 - Focused on producing correct and scalable data, but mostly from the platform's perspective
- Today
 - We shift **from producing data to serving data** as a product: making it usable, reliable, and efficient for consumers

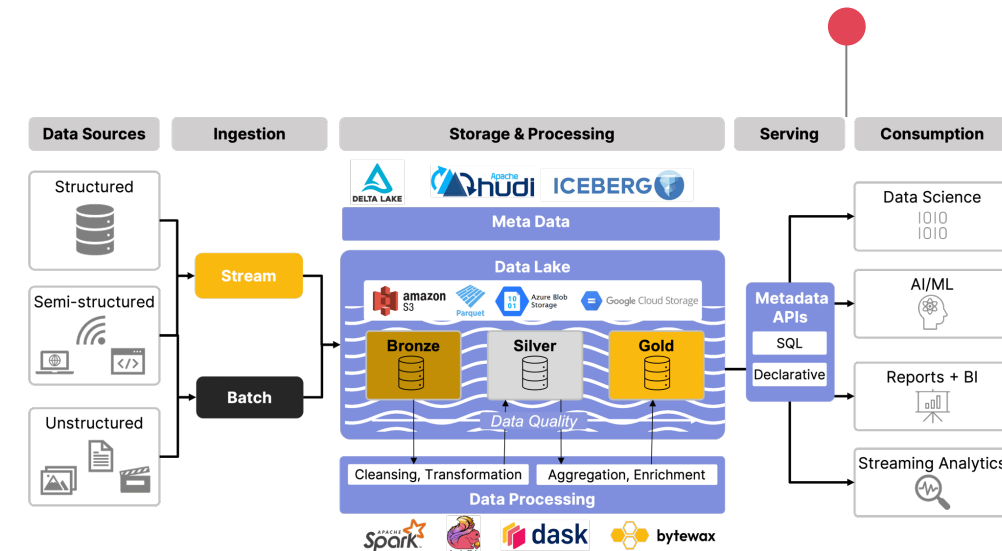


1. Serving as a product problem



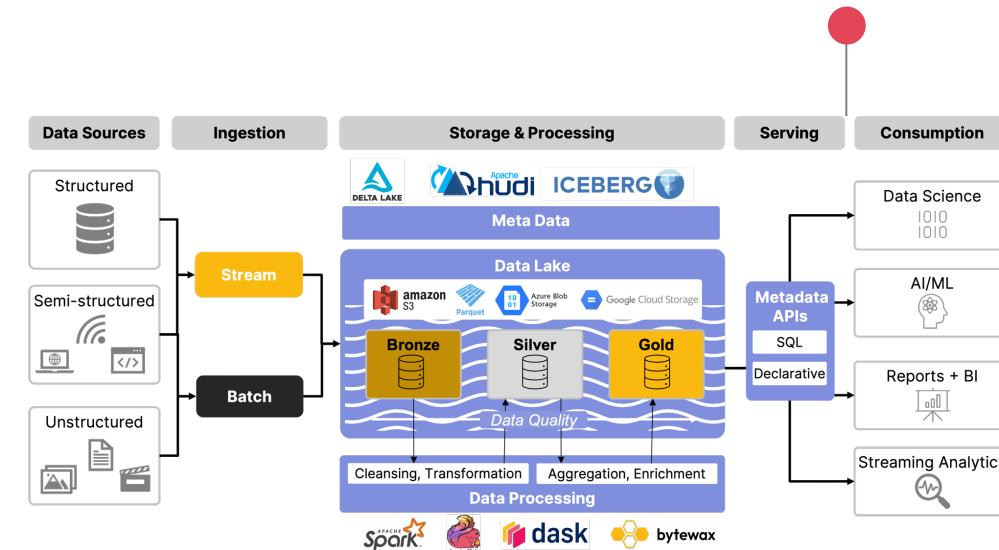
Gold is not the same as serving

- A Gold table is already business-oriented, but it is still often an internal platform artifact
- It may be correct and well-modeled, yet still be too slow, too complex, or too unstable for direct consumption
- Consumers usually do not want “a table”
 - They want an answer
 - They want it on time
 - They want it with consistent meaning
- So, Gold is a strong candidate for serving, but it is not serving by default



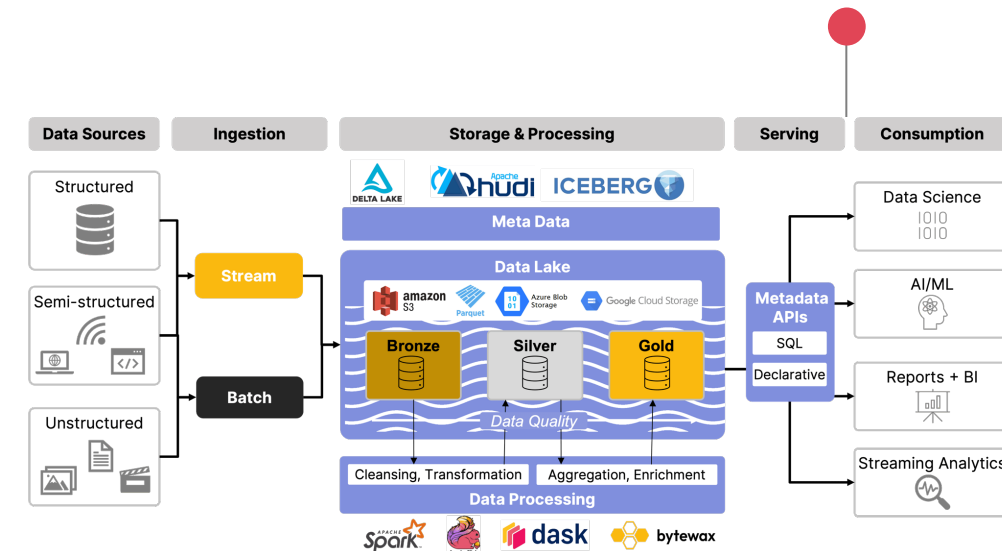
Consumers do not want tables, they want answers

- Different consumers expect different kinds of access (and have **different expectations**)
- Typical consumers in a data platform:
 - Dashboards / BI tools, expect fast, reusable, stable metrics
 - Analysts, expect flexible SQL access for exploration and slicing
 - Applications / APIs, expect predictable responses, low latency, controlled contracts
- The same underlying data product may need different serving interfaces for different consumers
- Therefore, serving starts by asking:
 - *"Who is the consumer, and what decision or action depends on this data?"*



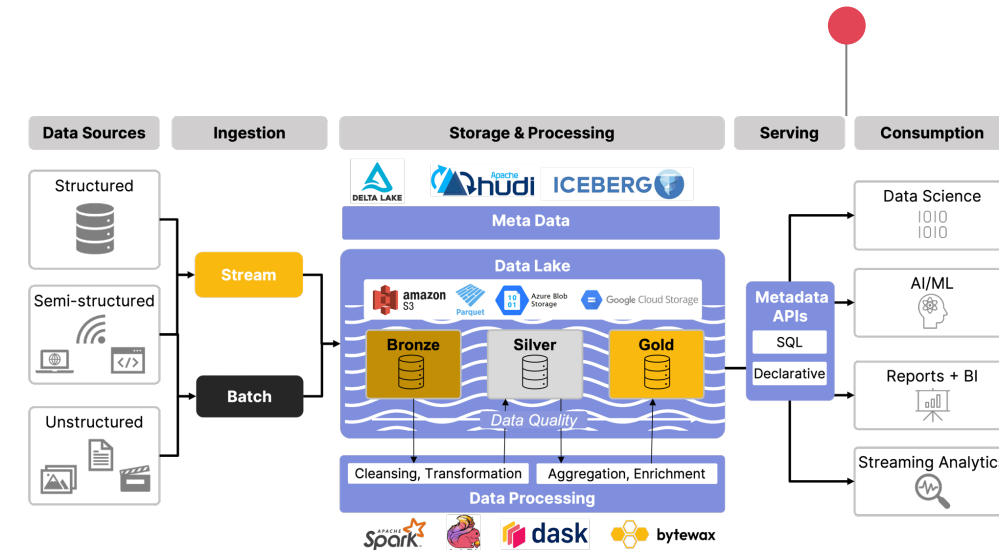
Serving has non-functional requirements

- To serve data well, **correctness is not enough**
- We must also think about
 - Latency
 - Milliseconds for APIs
 - Seconds for interactive dashboards
 - Minutes may still be acceptable for exploratory analysis
 - Concurrency
 - One analyst is different from 200 dashboard users at 09:00
 - Semantic stability
 - “revenue” must mean the same thing across consumers and over time
- A serving choice is always a trade-off between **flexibility, speed, cost, and stability**



Serving = interface + contract + predictable performance

- A serving layer is the point where a data product becomes usable
- In practice, good serving requires
 - An interface: SQL view, materialized table, or API
 - A contract: schema, semantics, versioning, expectations
 - Predictable performance: acceptable latency and behavior under load
- This is why serving is not just “exposing Gold”
 - It is the step where we transform platform outputs into consumer-ready data products
- A Gold table is a platform output. A served product is a consumer-facing interface



2. Data serving patterns and engines

There is more than one way to serve data

- Once data is modeled and available in the platform, we still need to decide how consumers will access it
- Common serving patterns include
 - Direct SQL access
 - Materialized tables / marts
 - APIs or application-facing services
- These patterns are not mutually exclusive
- The right choice depends on
 - Who consumes the data
 - How often it is queried
 - How fast it must respond
 - How stable the logic must be

Pattern 1: direct SQL serving

- Consumers query the data platform directly through SQL
- Typical examples
 - Analysts exploring data
 - BI tools querying views or tables
 - Internal teams reusing curated datasets
- Main strengths
 - Flexibility
 - Fast iteration
 - Low initial maintenance
- Main risks
 - Repeated expensive queries: when results could be computed only once and stored
 - Duplicated business logic: teams end up writing / duplicating code that was already written by someone else
 - Metric drift across teams: each team writes its own logic, and different concepts may be implemented differently
 - The concept of *Revenue* may be implemented as
 - `revenue = sum(paid_orders)` (BI)
 - `revenue = sum(paid_orders) - refunds` (Finance)
 - `revenue = sum(paid_orders, excluding test sales and promotions)` (Product)

Pattern 2: materialized serving

- The result is computed ahead of time and stored as a table ready for consumption
- Typical examples
 - Marts
 - KPI tables
 - Daily aggregates
 - Precomputed dashboard datasets
- Main strengths
 - Predictable latency
 - High reuse
 - More stable consumption
- Main risks
 - Refresh logic must be operated
 - Storage overhead increases
 - Freshness becomes an explicit responsibility

Pattern 3: API serving

- Data is exposed through an application-facing interface
- Typical examples
 - Endpoints for internal apps
 - Lightweight services answering business questions
 - External consumption with controlled access
- Main strengths
 - Clear contract
 - Consumer simplicity
 - Better control over access and usage
- Main risks
 - Stricter latency expectations
 - Versioning becomes more visible
 - Failures affect downstream applications directly

OLAP vs. OLTP: different serving needs

- Not all serving workloads are the same
- OLAP-style serving (analytical serving)
 - Analytical queries
 - Scans, aggregates, historical exploration
 - Usually seconds are acceptable
- OLTP-style serving (operational / transactional serving)
 - Small queries, point lookups, application interactions
 - Usually milliseconds are expected
- A lakehouse/query engine is excellent for many OLAP use cases
- It is not always the right direct interface for OLTP-like requirements...

This is a product decision, not just a technical one

- The right choice depends on the consumer context
 - How many times will this be used?
 - How fast must it respond?
 - How stable is the metric definition?
 - How fresh must the answer be?
- Typical rule of thumb
 - Evolving logic + few users > query-on-read
 - Repeated logic + many users + latency expectations > materialize
- In practice, many systems use a hybrid approach
 - Materialize the stable and expensive parts
 - Keep the final shaping flexible when needed

Where do we pay the cost?

- Every serving pattern answers the same question differently:
 - Do we pay the compute cost before consumption, during consumption, or both?
- Direct SQL
 - More flexibility
 - More cost and variability at query time
- Materialization
 - More preparation work
 - Lower and more predictable cost at consumption time
- APIs
 - Often rely on one of the previous two patterns underneath
 - But add a contract-oriented access layer
- In practice, most systems end up using a hybrid serving model
- Among these serving patterns, **SQL is often the default interface**. But being universal does not mean being sufficient...

SQL is the default interface for data

- In most data platforms, SQL is the primary way to access data
- It acts as a universal interface across
 - Query engines (e.g., Trino)
 - Data warehouses
 - BI tools
- Almost every consumer can interact with data through SQL
 - Analysts
 - Dashboards
 - Even APIs (indirectly)
- This makes SQL the “lingua franca” of data platforms
- SQL works so well because it is
 - Declarative: you describe what you want, not how to compute it
 - Expressive: supports filters, joins, aggregations, window functions
 - Flexible: allows exploration and rapid iteration
 - Standardized: widely known and supported across tools

But SQL alone is not enough...

- SQL gives access to data, but not necessarily
 - Consistent definitions
 - Controlled cost
 - Predictable performance
- Common issues in SQL-based serving
 - Duplicated logic across queries
 - Inconsistent metric definitions
 - Heavy queries executed repeatedly
- Different consumers may write different queries (not all of them correct or abiding to the same definitions...) for the same question

SQL also exposes cost and complexity

- Every SQL query has a real execution cost, which can be measured in terms of
 - Data scanned
 - Data shuffled
 - Memory used
- Complex queries can become
 - Slow
 - Expensive
 - Unpredictable under load
- What works for one analyst may not scale to
 - Many concurrent users
 - Production dashboards or APIs
- SQL serving is powerful, but not inherently safe for serving at scale
- It is most useful in early stages (closer to data science), not so much in later stages (engineering / production)

A query engine is a serving layer over the platform

- A query engine allows consumers to access data through a logical SQL interface
- It sits between the physical storage layer and the consumer-facing query layer
- In our stack, this means
 - Data lives in object storage / Iceberg tables
 - Consumers interact through Trino
- This decouples how data is stored from how data is consumed
- They also provide a single access layer for many consumers
- They work well for
 - Interactive analytics
 - BI tools
 - Reusable SQL views
 - Exploration over large datasets
- They reduce the need to move data into separate systems too early
- They make the lakehouse feel like a queryable product, not just stored files

Federated access: one interface, multiple sources

- Query engines can often expose multiple systems through the same SQL interface
- This means consumers may query
 - Lakehouse tables
 - Relational databases
 - Other external sources
- Main advantage
 - Consumers use one language and one entry point, although interacting with different sources
- Main caution
 - A unified interface does not eliminate semantic or performance differences across sources
 - It may even hide some of the relevant information from users (if they do not know with what they are interacting, they do not know how to optimize for it...)

A query engine is not always the final serving interface

- Query engines are excellent when consumers need
 - Flexible SQL
 - Analytical queries
 - Broad access to curated data
- But they are not always ideal for
 - Very low-latency use cases
 - Strict application-facing contracts
 - High-concurrency operational workloads
- In many systems, the query engine is
 - The primary analytical serving layer
 - But not the only serving layer
- Specifically, it does not replace
 - Semantic stability
 - Serving design decisions
 - Product-facing contracts
- Even when we choose the right serving pattern, one problem remains: how do we ensure that everyone means the same thing by the same metric?

3. Semantic layer

Metrics, dimensions, and creating a single source of truth

Access is not the same as meaning

- A serving layer may expose data correctly, but that does not guarantee semantic consistency
- Two consumers may access the same Gold data and still produce different answers
- The core problem is not only access
- It is also
 - What each metric means
 - What each row represents
 - Which dimensions are valid for slicing
- So, even with good tables, we still need a way to stabilize meaning

What is a semantic layer (in practice)?

- A semantic layer is a **central repository** where metrics and dimensions are defined once and reused
- It sits on top of Gold data
- It is typically implemented as
 - SQL views (in Trino, in our case)
 - Or a metrics definition layer (in more advanced setups)
- It exposes
 - Named metrics (e.g., net_revenue_daily)
 - Allowed dimensions (e.g., country, date)
- Consumers query these definitions, not raw tables

From Gold tables to semantic views

- Gold tables contain the data
- The semantic layer defines how to interpret that data

- Example:

```
-- gold.orders
-- (order_id, customer_id, amount_eur, status, order_date)

-- semantic view
CREATE VIEW metrics.daily_revenue AS
SELECT
  order_date,
  SUM(amount_eur) AS net_revenue
FROM gold.orders
WHERE status = 'paid'
GROUP BY order_date;
```

- Consumers now query `SELECT * FROM metrics.daily_revenue;`

What do we define in a semantic layer?

- A semantic layer encodes:
 - Metric definitions, such as formulas (e.g., sum, count, ratio)
 - Filters / business rules (e.g., exclude cancelled orders)
 - Time logic: daily, rolling windows, timezone
 - Grain: per day, per customer, per order
 - Dimensions: what can we group by safely?
- These definitions should **match the data contract semantics**

The challenge is not only technical

- Having a semantic layer does not guarantee that people will use it correctly
- If there is no documentation
 - No one knows which metrics exist
 - No one knows which to use when
 - No one knows what things mean
- There are 3 properties we must ensure in a semantic layer
 - Discoverable: existing products can be searched / discovered by users
 - Understandable: users must understand what a certain product / metric means and what it measures
 - Usable: users must understand right away how to access the data, maybe with an example
- Nowadays, at a time in which we interact increasingly more with data through **LLMs / agents** and in more dynamic ways, a proper semantic layer is paramount to ensure agents (as well as humans) can discover and understand the meaning of each artifact

How to document

- Creating / documenting a semantic layer starts with a consistent naming structure
- A typical convention is
 - It starts with the prefix `metrics`.
 - It has an explicit name
 - The granularity is included in the name
- Examples
 - `metrics.daily_net_revenue`
 - `metrics.active_customers_30d`
 - `metrics.conversion_rate_daily`

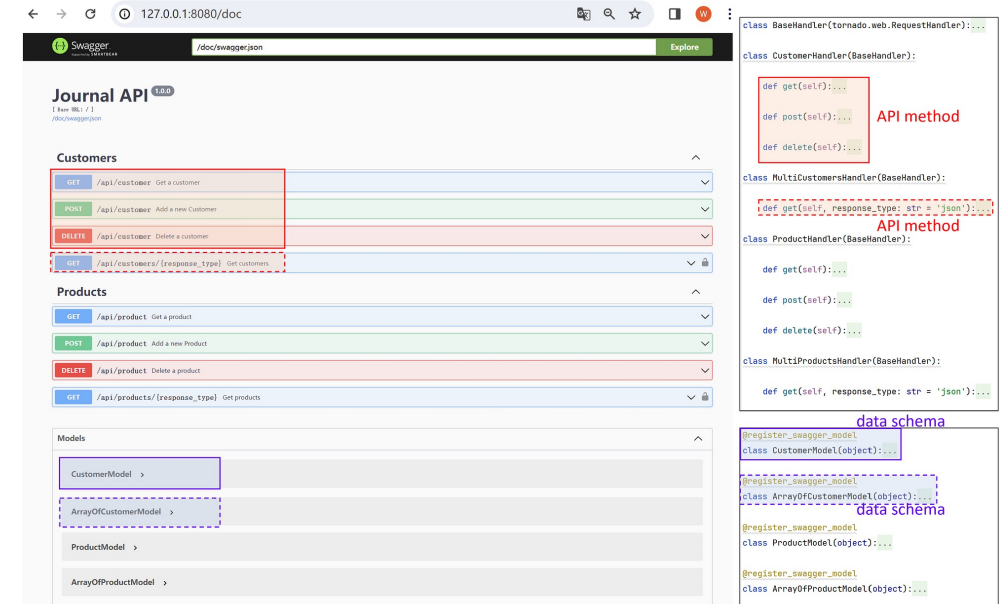
How to document

- Each view, API, etc. must be self-documented
 - Comments for SQL
 - Documentation (e.g. Swagger) for APIs
 - YAML files

-- Metric: daily_net_revenue
-- Grain: 1 row per day (UTC)
-- Definition:
-- sum of paid orders minus refunds recognized within 7 days
-- Filters:
-- exclude cancelled and test orders
-- Owner: data_team

```
CREATE VIEW metrics.daily_net_revenue AS
SELECT
  order_date,
  SUM(amount_eur - refund_eur) AS net_revenue
FROM gold.orders
WHERE status = 'paid'
GROUP BY order_date;
```

```
metrics:
- name: daily_net_revenue
  description: Net revenue per day in EUR
  grain: day
  formula: sum(paid_orders) - refunds (T+7)
  dimensions:
    - date
    - country
  owner: data_team
  source: metrics.daily_net_revenue
```



How to document

- Each view, API, etc. must be self-documented
 - Comments for SQL
 - Documentation (e.g. Swagger) for APIs
 - YAML files

```
-- Metric: daily_net_revenue
-- Grain: 1 row per day (UTC)
-- Definition:
--   sum of paid orders minus refunds recognized within 7 days
-- Filters:
--   exclude cancelled and test orders
-- Owner: data_team
```

```
CREATE VIEW metrics.daily_net_revenue AS
SELECT
  order_date,
  SUM(amount_eur - refund_eur) AS net_revenue
FROM gold.orders
WHERE status = 'paid'
GROUP BY order_date;
```

```
metrics:
- name: daily_net_revenue
  description: Net revenue per day in EUR
  grain: day
  formula: sum(paid_orders) - refunds (T+7)
  dimensions:
    - date
    - country
  owner: data_team
  source: metrics.daily_net_revenue
```

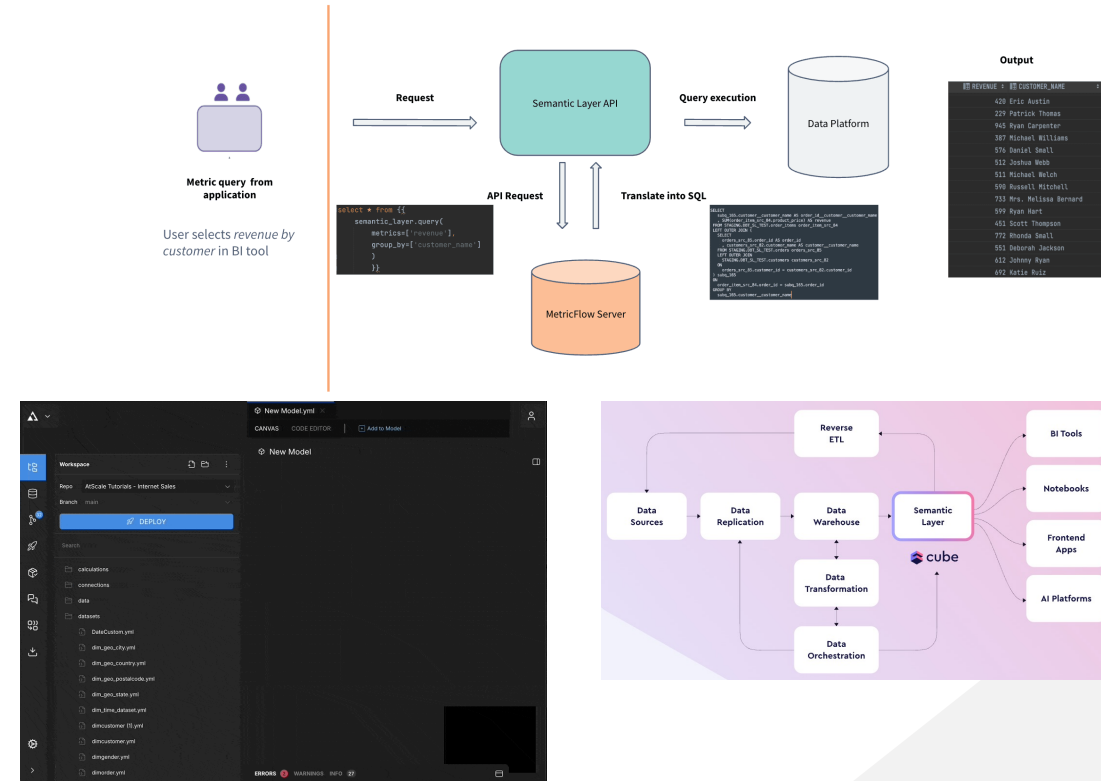
The screenshot displays a Swagger API documentation interface. The browser window at the top shows the URL '127.0.0.1:8080/doc'. The Swagger logo is visible in the top left corner. The 'Models' section is expanded, showing the 'CustomerModel' and 'ArrayOfCustomerModel' definitions. The 'CustomerModel' definition includes a description 'Customer model representation' and a JSON schema with fields: 'id' (integer), 'name' (string), 'is_visible' (boolean), and 'data' (array of objects). The 'ArrayOfCustomerModel' definition includes a description 'Array of customer model representation' and a JSON schema with fields: 'response_type' (string) and 'data' (array of 'CustomerModel' objects). To the right of the Swagger interface, there are two code blocks labeled 'data schema' in red and 'data schema' in purple. The red block shows a Python class definition for 'CustomerModel' with a '@register_swagger_model' decorator. The purple block shows a Python class definition for 'ArrayOfCustomerModel' with a '@register_swagger_model' decorator.

Catalogs

- There should be a central repository, that describes all the metrics and how to access them
- It should state, at least, what exists, what does it mean, and how it can be accessed
- The catalog can be
 - A document (e.g. readme) with a list of metrics, their description, how to use them, and examples of provided data
 - A list of the SQL views: `SHOW TABLES FROM metrics;`
 - A guide with practical examples on how to use
 - A specific tool
 - ...
- Things to avoid
 - Views without descriptions or documentation
 - Generic naming (e.g. `kpi_table_final_v2`)
 - Metrics that are accessible only on dashboards
 - Hidden logic in BI queries
 - No owner

Tools (for more structured approaches)

- dbt Semantic Layer / MetricFlow
 - Defines metrics in code on top of dbt models and exposes them through multiple interfaces. The semantic layer centralizes metric definitions and allows them to be queried consistently from downstream tools
- Cube
 - Acts as a semantic layer exposed via SQL APIs and other interfaces, providing a single source of truth for metrics used by dashboards and applications
- Looker / LookML
 - LookML is Looker's modeling language for defining semantic models. Documentation, exploration, and consumption are unified within the same platform
- AtScale
 - Positions itself as a universal semantic layer, exposing metrics and business logic consistently across BI tools, analytical applications, and other consumers



MCP: serving data to agents (LLMs)

- Up until now, we should serve data mostly to:
 - Human users, mostly through SQL
 - Applications, mostly through REST APIs
- Now, increasingly, we must also serve data for AI agents (LLMs)
- MCP (Model Context Protocol) defines a way for LLMs to access external tools and data
- An MCP server can expose
 - SQL queries
 - Metrics (semantic layer)
 - Data APIs
- This turns data products into agent-accessible interfaces

```
from mcp.server import Server

server = Server()

@server.tool
def list_metrics():
    """List available metrics"""
    return ["daily_net_revenue", "active_customers_30d"]

@server.tool
def query_metric(metric_name: str, country: str):
    """Query a metric for a given country"""

    query = f"""
    SELECT *
    FROM metrics.{metric_name}
    WHERE country = '{country}'
    """

    # running the query on trino
    results = run_trino_query(query)

    return results

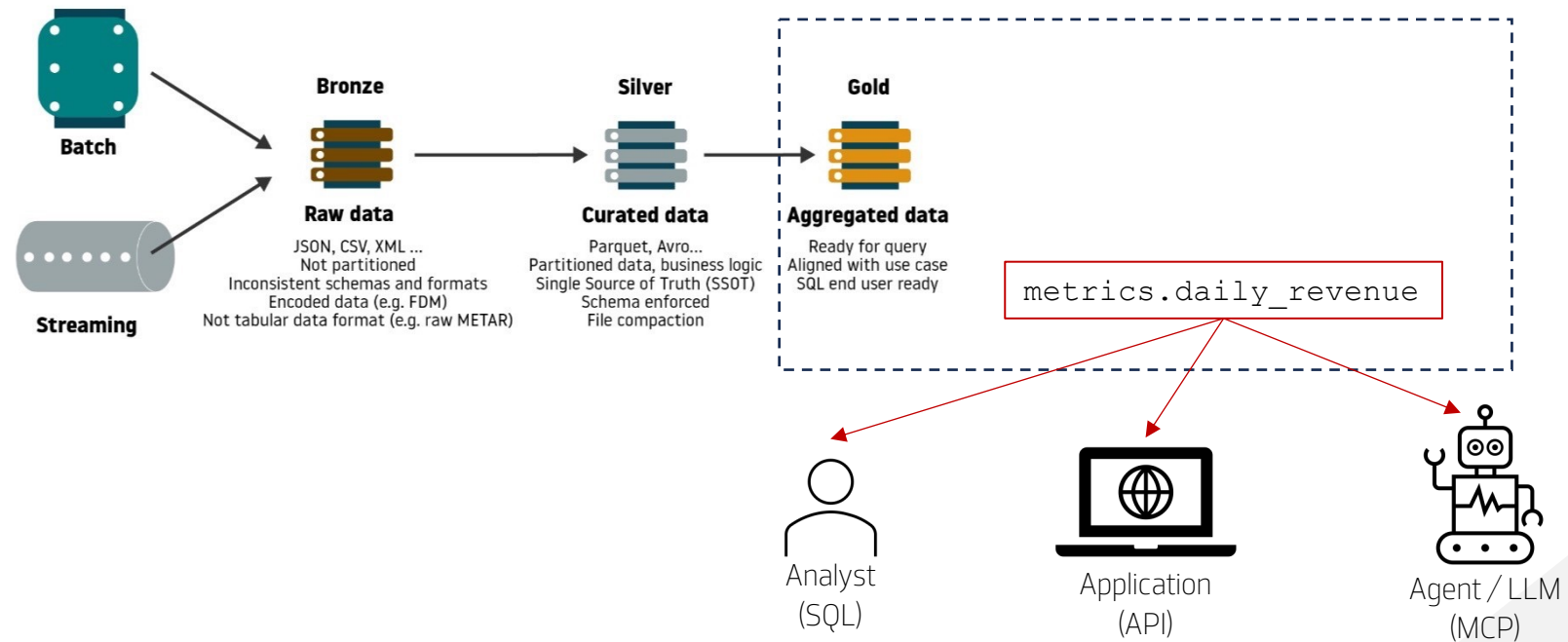
if __name__ == "__main__":
    server.run()
```

Why not just query Gold directly?

- Without a semantic layer
 - Each consumer writes their own logic
 - Definitions diverge over time
- Typical problems
 - Same metric computed differently
 - Duplicated SQL across dashboards
 - Harder debugging and maintenance
- With a semantic layer
 - Logic is defined once
 - Reused everywhere
 - Easier to evolve safely

Multiple interfaces for the same product

- As the uses of the same data increase, it becomes increasingly common to expose multiple interfaces for the same product (eventually showing variants of that product)



4. APIs and Data Products

When SQL is not enough

- Not all consumers can or should use SQL
- Common cases
 - Applications (frontend, backend services)
 - External integrations
 - Automated systems
- These consumers need
 - Simple interfaces
 - Predictable responses
 - Low latency
- This is where APIs become the natural serving interface

What is a data API?

- A data API exposes data through explicit endpoints
- Each endpoint defines
 - Inputs (parameters)
 - Outputs (structured response)
- Example
 - `GET /daily-revenue?country=PT&start=2026-03-01&end=2026-03-07`
- The consumer does not need to know
 - Table names
 - Joins
 - Business logic
 - ...
- APIs should not reimplement or add new business logic
- Instead, they should reuse semantic views (metrics.*)
- This ensures
 - Consistent definitions
 - Reuse of logic
 - Easier maintenance

```
[  
  { "date": "2026-03-01", "revenue": 12450.80 },  
  { "date": "2026-03-02", "revenue": 11890.25 }  
]
```


Designing good data APIs

- Good APIs expose business concepts, not raw data
 - Prefer: `GET /daily-revenue?country=PT`
 - Avoid: `GET /query?sql=SELECT * FROM ...`
- Good practices
 - Clear naming (`/daily-revenue`, `/top-customers`)
 - Explicit parameters
 - Controlled outputs
 - Predictable behavior

Trade-offs of API serving

- Advantages
 - Simple for consumers
 - Controlled access
 - Stable and reusable interface
- Trade-offs
 - Less flexibility than SQL
 - Requires design and maintenance
 - Versioning becomes necessary
- APIs are best suited for
 - Repeated use cases
 - Application integration
 - Stable metrics

Scalable Technologies for Data Analysis

Serving

Davide Carneiro

Mestrado em Engenharia Informática

2025/2026